Práctica 1: Agentes en "La Red" - Navegación en Tron

1. Introducción

El objetivo de esta práctica es implementar agentes inteligentes capaces de navegar en un entorno con restricciones dinámicas. La temática se basa en el universo de **TRON**: el agente es una "Moto de Luz" que debe atravesar la "Red" (Grid) desde una posición inicial hasta un **Portal de Salida (S)**.

A diferencia de un laberinto clásico, en este entorno **el movimiento tiene consecuencias físicas**: el rastro que deja el agente se solidifica (representado por el carácter "`.`"), convirtiéndose en una nueva pared. Esto significa que **no se puede retroceder físicamente** sobre los propios pasos.

2. Arquitectura del Código Base

Se proporciona un único archivo `Tron_P1_BaseSimple.java` que contiene todo lo necesario para la simulación. El código está dividido en tres partes:

1. **Clase `Tron_P1_BaseSimple` (Main):** Contiene el bucle de simulación.
2. **Clase `Entorno`:** El motor del juego (Física y Reglas).
3. **Clases de Agentes:** Donde el alumno debe trabajar.

2.1. El Motor del Juego: Clase `Entorno`

Esta clase ya está implementada y **no debéis modificarla**. Funciona como la "realidad" que percibe el agente. Debéis entender sus métodos para que vuestro agente pueda "ver".

Variables Importantes:

- `grid`: Matriz de caracteres que representa el mapa.
- `agenteF`, `agenteC`: Coordenadas enteras (Fila, Columna) de donde está el agente actualmente.

Métodos de Percepción (Sensores): El agente usará estos métodos para decidir si puede moverse o no.

- `boolean esTransitable(int f, int c)`:
  - Devuelve **`true`** si la casilla está libre (es un espacio vacío `' '` o la meta `'S'`).
  - Devuelve **`false`** si hay un obstáculo: Muro (`#`), Rastro (`.`) o si las coordenadas están fuera de los límites del mapa.
- `boolean hemosGanado()`: Indica si el agente ha llegado a la salida.

Método de Actuación (Actuadores):

- `moverAgente(String accion)`: Recibe "N", "S", "E", u "O". Actualiza la posición del agente en el grid y deja el rastro (`.`) en la posición anterior.

---

3. Tareas a Realizar

Vuestro objetivo es implementar la inteligencia ("método `pensar`") de dos tipos de agentes.

Tarea 0: El Agente Interactivo (Familiarización)

Antes de programar inteligencias, usaréis este agente para jugar manualmente.

- **Objetivo:** Entender cómo funciona el bloqueo por estelas.
- **Funcionamiento:** El programa os pedirá por consola una dirección (N, S, E, O). Si el movimiento es válido, el agente avanza. Si hay pared o estela, se queda en el sitio y pierde el turno.

Tarea 1: El Agente Reactivo

El **Agente Reactivo** toma decisiones basándose únicamente en el estado actual del mundo. No tiene memoria de dónde ha estado antes (aparte de lo que ve físicamente en el grid).

Requisitos:

1. En la clase `AgenteReactivo`, completad el método `pensar(Entorno mapa)`.
2. Obtened la posición actual del agente usando `mapa.agenteF` y `mapa.agenteC`.
3. Utilizad `mapa.esTransitable(f, c)` para comprobar las 4 direcciones (Norte, Sur, Este, Oeste).
4. Cread una lista con las acciones válidas y devolved una aleatoria.
5. **Observación:** Notaréis que este agente tiende a encerrarse en callejones sin salida o chocarse consigo mismo fácilmente.

Tarea 2: El Agente Deliberativo: La estrategia

Tarea 2.A El agente que podía ver el futuro

Este agente puede consultar en el mapa hasta dos posiciones más allá de su posición actual. Gráficamente, en el ejemplo, las casillas en negrita son las alcanzables.

```
_ _ _ _ _
_ _ _ _ _
_ _ A _ _
_ _ _ _ _
_ _ _ _ _
```

Lo que da 8 opciones más a las cuatro iniciales.

Estrategias:

- Si una opción lleva en una o dos jugadas a la S, elegir esa directamente.
- Priorizar aquellas las que lleven a más huecos: la acción "E" no tiene huecos disponibles (es una muerte segura), la acción "N" tiene 1 hueco disponible a la Izquierda, la Opción "S", no existe porque no es transitable, la opción "O" tiene 3 huecos disponibles, por lo que el orden de las posibles es O : 3 huecos, N : 1 huecos , E :0 huecos , Acción elegida→ O . Si hay más de una con el mismo número de huecos se debe aleatorizar entre ellas.

```
_ _ # _ _
_ _ _ # _
_ _ A _ #
_ _ . # _
_ _ . _ _
```

Tarea 2.B El agente que se acordaba de todo

Esto le permite recordar casillas que ha visitado o estado a cerca pudiendo realizar estrategias de movimiento.

Implementación de la Memoria: En este código simplificado, la memoria se gestiona mediante una matriz de contenido similar al grid, pero que inicialmente está vacía.

Requisitos:

1. En el constructor `AgenteDeliberativo`, inicializad la matriz `memoria` con el tamaño del mapa (`filas` x `cols`).
2. En el método `pensar`:
    - Marcad la posición actual en vuestra memoria: `memoria[f][c] = ".";`.
    - Analizad los vecinos (N, S, E, O).
    - Guardar contenido en esas posiciones en la memoria interna
    - Un vecino es interesante si: Es transitable en el mundo real
    - Establecer una cola de prioridad con las casillas transitables según algún criterio, por ejemplo:
        1. Mantener inercia (acordarme de la dirección que llevaba)
        2. Buscar zonas libres de nuestro rastro. Si la dirección en la que se mueve en memoria en línea recta contiene un rastro, ya hemos estado allí. Priorizar por el número de casillas antes de llegar a limite en esa dirección (#, o fin de mapa)
        3. Etc..

Tarea 2.C El agente que lo sabía todo

Al inicializar carga en su memoria el grid completo y puede decidir en base a esta información. Implementar que la casilla más cercana a la salida que sea transitable sea la próxima acción.

4. Ejecución y Pruebas

A) Pruebas de Arranque: Para probar vuestro código, id al método `main` y seguid estos pasos:

1. **Configurar el Mapa:** Podéis editar la variable `mapaTexto` para crear laberintos más difíciles. Probad con el pequeño y el grande.

```
static final String MAPA_PEQUENO =
        "#########\n" +
        "#A      #\n" +
        "#  #### #\n" +
        "#       #\n" +
        "#     S #\n" +
        "#########";

static final String MAPA_GRANDE =
        "####################\n" +
        "#A  #              #\n" +
        "# # # ###### ##### #\n" +
        "# #   #      #     #\n" +
        "# ##### ###### ### #\n" +
        "#     #        #   #\n" +
        "# ### ######## # # #\n" +
        "#   #          #   #\n" +
        "# ############## S #\n" +
        "####################"
```

2. **Seleccionar el Agente:** Descomentad la línea del agente que queréis evaluar.

```
AgenteInteractivo agente = new AgenteInteractivo();


// AgenteReactivo agente = new AgenteReactivo()
 //AgenteDeliberativo agente = new AgenteDeliberativo();
```

3. **Ejecutar:** Observad la salida por consola. El programa imprimirá el grid paso a paso.

4. Rúbrica de Evaluación y entrega

Deberá entregarse en el link del grupo al que se pertenezca (IA1- IA4) hasta el 20 de marzo.
En las dos siguientes clases se citará en orden alfabético al alumno en clase para defender oralmente la misma. Si esta no es correcta se podrá reentregar en segunda convocatoria.

Suspenso: El código no compila o los agentes atraviesan paredes/rastros (ignoran `esTransitable`). El agente reactivo no funciona.

- **Aprobado (5-6):** El Agente Reactivo funciona correctamente, moviéndose aleatoriamente sin chocar de inmediato. Se implementa el agente con conocimiento de salida (2c)
- **Notable (7-8):** Lo anterior+ El Agente Deliberativo 2B implementa correctamente la matriz de memoria. El agente prioriza la exploración de casillas usando la memoria. Se incluyen pruebas de la ejecución en la memoria. Se ha implementado alguna estrategia adicional a las propuestas al agente deliberativo. Se muestran diversas ejecuciones con laberintos que muestran esa estrategia.
- **Sobresaliente (9):** lo anterior + el código es limpio y eficiente. El alumno ha probado mapas complejos con pruebas que muestran y en el informe explica la diferencia de comportamiento del reactivo y el deliberativo. Se implementa el agente deliberativo 2A
- **Sobresaliente (10):** lo anterior + se implementa, explica y muestran salidas del de los agentes.

Anexo: Código Base (`Tron_P1_BaseSimple.java`)

Este es el código que debéis utilizar. Copiadlo íntegramente en vuestro IDE (Eclipse/IntelliJ) y trabajad sobre las clases `AgenteReactivo` y `AgenteDeliberativo`.

Java
```
import java.util.*;

public class Tron_P1_BaseSimple {

    public static void main(String[] args) throws InterruptedException
{

        // Podéis editar este mapa para las pruebas
        String mapaTexto =
                "########\n" +
                "#A     #\n" +
                "#      #\n" +
                "#    S #\n" +
                "########";

        // 1. Inicializar Entorno
        Entorno juego = new Entorno(mapaTexto);
```

```
        // 2. Elegir Agente (Descomentar el que se quiera probar)
        // AgenteReactivo agente = new AgenteReactivo();
        AgenteDeliberativo agente = new
AgenteDeliberativo(juego.filas, juego.cols);

        System.out.println("--- Inicio de la Simulación ---");
        int ciclos = 0;
        boolean juegoActivo = true;

        // BUCLE DE SIMULACIÓN
        while (juegoActivo && ciclos < 200) {
            juego.dibujar();

            // El agente decide
            String accion = agente.pensar(juego);

            // El entorno aplica el movimiento
            boolean movimientoExitoso = juego.moverAgente(accion);

            System.out.println("Ciclo " + ciclos + " | Acción: " +
accion);

            if (!movimientoExitoso) {
                System.out.println(">>> ¡CRASH! El agente se ha
chocado.");
                juegoActivo = false;
            } else if (juego.hemosGanado()) {
                System.out.println(">>> ¡VICTORIA! Salida
encontrada.");
                juego.dibujar();
                juegoActivo = false;
            }

            Thread.sleep(300); // Retardo para animación
            ciclos++;
        }
    }
}

/* ======================================================
   CLASE ENTORNO (INFRAESTRUCTURA)
   Gestiona la matriz del mapa y las reglas físicas.
   NO MODIFICAR.
   ====================================================== */
class Entorno {
    char[][] grid;
    int filas, cols;
    int agenteF, agenteC; // Coordenadas del agente
    int metaF, metaC;     // Coordenadas de la meta

    public Entorno(String mapa) {
        String[] lineas = mapa.trim().split("\n");
        filas = lineas.length;
        cols = lineas[0].length();
        grid = new char[filas][cols];

        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < cols; j++) {
                char celda = lineas[i].charAt(j);
                grid[i][j] = celda;
```

```
                if (celda == 'A') {
                    agenteF = i; agenteC = j;
                }
                if (celda == 'S') {
                    metaF = i; metaC = j;
                }
            }
        }
    }

    // Comprueba si una casilla es segura para moverse
    public boolean esTransitable(int f, int c) {
        if (f < 0 || f >= filas) return false;
        if (c < 0 || c >= cols) return false;
        char celda = grid[f][c];
        // Es transitable si no es Muro (#) ni Rastro (.)
        return celda != '#' && celda != '.';
    }

    public boolean esMeta(int f, int c) {
        return f == metaF && c == metaC;
    }

    public boolean hemosGanado() {
        return esMeta(agenteF, agenteC);
    }

    // Aplica el movimiento y deja el rastro
    public boolean moverAgente(String accion) {
        int sigF = agenteF;
        int sigC = agenteC;

        if (accion.equals("N")) sigF--;
        if (accion.equals("S")) sigF++;
        if (accion.equals("E")) sigC++;
        if (accion.equals("O")) sigC--;

        if (!esTransitable(sigF, sigC)) return false;

        // Dejar rastro en la posición anterior
        grid[agenteF][agenteC] = '.';

        // Actualizar posición
        agenteF = sigF;
        agenteC = sigC;

        if (!esMeta(agenteF, agenteC)) {
            grid[agenteF][agenteC] = 'A';
        }
        return true;
    }

    public void dibujar() {
        System.out.println("----------");
        for (int i = 0; i < filas; i++) {
            System.out.println(new String(grid[i]));
        }
    }
}

/* ====================================================
```

```
   ZONA DEL ALUMNO: AGENTES
   ================================================== */

class AgenteReactivo {
        // --- INICIO CODIGO ALUMNO ---
        // ...
        // --- FIN CODIGO ALUMNO ---
    public String pensar(Entorno mapa) {

        // TAREA 1: Completar lógica reactiva
        // Usar mapa.esTransitable(...) para mirar N, S, E, O
        // Añadir a 'opciones' las direcciones válidas

        // --- INICIO CODIGO ALUMNO ---
        // ...
        // --- FIN CODIGO ALUMNO ---

        return accion. //"N" "S" "E" "0"
    }
}

class AgenteDeliberativo {


    public AgenteDeliberativo(int filas, int cols) {
        memoria = new xxxxx[filas][cols];
    }

    public String pensar(Entorno mapa) {
;

        // TAREA 2: Implementar lógica deliberativa
        // 1. Mirar vecinos.
        // 2. Si esTransitable -> añadir a opcionesSeguras.
        // 3. Valorar opciones sobre la memoria

        // --- INICIO CODIGO ALUMNO ---
        // ...
        // --- FIN CODIGO ALUMNO ---

        // Prioridad: transitable, movimiento favorable …

        }
        return …//"acción más favorable"
    }
}
```